

the end-effector orientation as $\phi = \vartheta_1 + \vartheta_2 + \vartheta_3$ coincides with the matrix that is obtained by eliminating the three null rows of the geometric Jacobian.

3.7 Inverse Kinematics Algorithms

In Sect. 3.5 it was shown how to invert kinematics by using the differential kinematics equation. In the numerical implementation of (3.48), computation of joint velocities is obtained by using the inverse of the Jacobian evaluated with the joint variables at the previous instant of time

$$\mathbf{q}(t_{k+1}) = \mathbf{q}(t_k) + \mathbf{J}^{-1}(\mathbf{q}(t_k))\mathbf{v}_e(t_k)\Delta t.$$

It follows that the computed joint velocities $\dot{\mathbf{q}}$ do not coincide with those satisfying (3.47) in the continuous time. Therefore, reconstruction of joint variables $\mathbf{q}$ is entrusted to a numerical integration which involves *drift* phenomena of the solution; as a consequence, the end-effector pose corresponding to the computed joint variables differs from the desired one.

This inconvenience can be overcome by resorting to a solution scheme that accounts for the *operational space error* between the desired and the actual end-effector position and orientation. Let

$$\mathbf{e} = \mathbf{x}_d - \mathbf{x}_e \quad (3.67)$$

be the expression of such error.

Consider the time derivative of (3.67), i.e.,

$$\dot{\mathbf{e}} = \dot{\mathbf{x}}_d - \dot{\mathbf{x}}_e \quad (3.68)$$

which, according to differential kinematics (3.62), can be written as

$$\dot{\mathbf{e}} = \dot{\mathbf{x}}_d - \mathbf{J}_A(\mathbf{q})\dot{\mathbf{q}}. \quad (3.69)$$

Notice in (3.69) that the use of operational space quantities has naturally lead to using the analytical Jacobian in lieu of the geometric Jacobian. For this equation to lead to an *inverse kinematics algorithm*, it is worth relating the computed joint velocity vector $\dot{\mathbf{q}}$ to the error $\mathbf{e}$ so that (3.69) gives a differential equation describing error evolution over time. Nonetheless, it is necessary to choose a relationship between $\dot{\mathbf{q}}$ and $\mathbf{e}$ that ensures convergence of the error to zero.

Having formulated inverse kinematics in algorithmic terms implies that the joint variables $\mathbf{q}$ corresponding to a given end-effector pose $\mathbf{x}_d$ are accurately computed only when the error $\mathbf{x}_d - \mathbf{k}(\mathbf{q})$ is reduced within a given threshold; such settling time depends on the dynamic characteristics of the error differential equation. The choice of $\dot{\mathbf{q}}$ as a function of $\mathbf{e}$ permits finding inverse kinematics algorithms with different features.

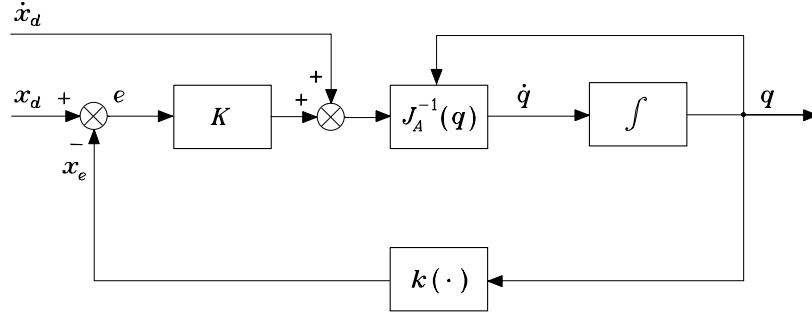


Fig. 3.11. Inverse kinematics algorithm with Jacobian inverse

3.7.1 Jacobian (Pseudo-)inverse

On the assumption that matrix $\mathbf{J}_A$ is square and nonsingular, the choice

$$\dot{\mathbf{q}} = \mathbf{J}_A^{-1}(\mathbf{q})(\dot{\mathbf{x}}_d + \mathbf{K}\mathbf{e}) \quad (3.70)$$

leads to the equivalent linear system

$$\dot{\mathbf{e}} + \mathbf{K}\mathbf{e} = \mathbf{0}. \quad (3.71)$$

If $\mathbf{K}$ is a positive definite (usually diagonal) matrix, the system (3.71) is *asymptotically stable*. The error tends to zero along the trajectory with a convergence rate that depends on the eigenvalues of matrix $\mathbf{K}$; ¹¹ the larger the eigenvalues, the faster the convergence. Since the scheme is practically implemented as a discrete-time system, it is reasonable to predict that an upper bound exists on the eigenvalues; depending on the sampling time, there will be a limit for the maximum eigenvalue of $\mathbf{K}$ under which asymptotic stability of the error system is guaranteed.

The block scheme corresponding to the inverse kinematics algorithm in (3.70) is illustrated in Fig. 3.11, where $\mathbf{k}(\cdot)$ indicates the direct kinematics function in (2.82). This scheme can be revisited in terms of the usual feedback control schemes. Specifically, it can be observed that the nonlinear block $\mathbf{k}(\cdot)$ is needed to compute $\mathbf{x}$ and thus the tracking error $\mathbf{e}$, while the block $\mathbf{J}_A^{-1}(\mathbf{q})$ has been introduced to compensate for $\mathbf{J}_A(\mathbf{q})$ and making the system linear. The block scheme shows the presence of a string of integrators on the forward loop and then, for a constant reference ($\dot{\mathbf{x}}_d = \mathbf{0}$), guarantees a null steady-state error. Further, the *feedforward* action provided by $\dot{\mathbf{x}}_d$ for a time-varying reference ensures that the error is kept to zero (in the case $\mathbf{e}(0) = \mathbf{0}$) along the whole trajectory, independently of the type of desired reference $\mathbf{x}_d(t)$.

Finally, notice that (3.70), for $\dot{\mathbf{x}}_d = \mathbf{0}$, corresponds to the Newton method for solving a system of nonlinear equations. Given a constant end-effector pose $\mathbf{x}_d$, the algorithm can be keenly applied to compute one of the admissible

¹¹ See Sect. A.5.

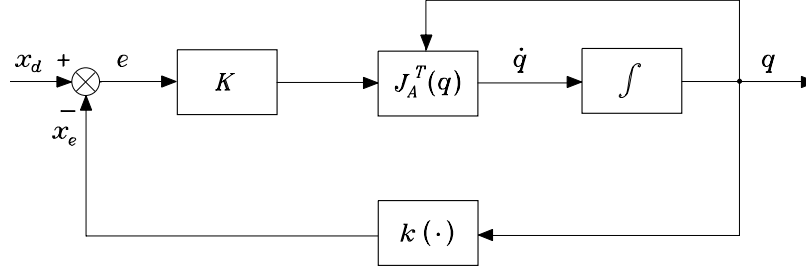


Fig. 3.12. Block scheme of the inverse kinematics algorithm with Jacobian transpose

solutions to the inverse kinematics problem, whenever that does not admit closed-form solutions, as discussed in Sect. 2.12. Such a method is also useful in practice at the start-up of the manipulator for a given task, to compute the corresponding joint configuration.

In the case of a *redundant manipulator*, solution (3.70) can be generalized into

$$\dot{\mathbf{q}} = \mathbf{J}_A^\dagger(\dot{\mathbf{x}}_d + \mathbf{K}\mathbf{e}) + (\mathbf{I}_n - \mathbf{J}_A^\dagger \mathbf{J}_A)\dot{\mathbf{q}}_0, \quad (3.72)$$

which represents the algorithmic version of solution (3.54).

The structure of the inverse kinematics algorithm can be conceptually adopted for a simple robot control technique, known under the name of *kinematic control*. As will be seen in Chap. 7, a manipulator is actually an electro-mechanical system actuated by motor torques, while in Chaps. 8–10 dynamic control techniques will be presented which will properly account for the non-linear and coupling effects of the dynamic model.

At first approximation, however, it is possible to consider a kinematic command as system input, typically a velocity. This is possible in view of the presence of a low-level control loop, which ‘ideally’ imposes any specified reference velocity. On the other hand, such a loop already exists in a ‘closed’ control unit, where the user can also intervene with kinematic commands. In other words, the scheme in Fig. 3.11 can implement a kinematic control, provided that the integrator is regarded as a simplified model of the robot, thanks to the presence of single joint local servos, which ensure a more or less accurate reproduction of the velocity commands. Nevertheless, it is worth underlining that such a kinematic control technique yields satisfactory performance only when one does not require too fast motions or rapid accelerations. The performance of the independent joint control will be analyzed in Sect. 8.3.

3.7.2 Jacobian Transpose

A computationally simpler algorithm can be derived by finding a relationship between $\dot{\mathbf{q}}$ and $\mathbf{e}$ that ensures error convergence to zero, without requiring linearization of (3.69). As a consequence, the error dynamics is governed by a